

Machine Learning Lab 4 Report

PB22111599杨映川

1 实验流程

1.1 Value Iteration

使用computeQ函数，根据当前的V(s)计算Q(s, a)

```
1     def computeQ(V: Dict[StateT, float], state: StateT, action: ActionT) ->
float:
2         # Return Q(state, action) based on V(state)
3
4         # BEGIN_YOUR_CODE
5         # raise Exception("Not implemented yet")
6         q_value = 0.0
7         for next_state, prob, reward in succAndRewardProb.get((state,
action), []):
8             q_value += prob * (reward + discount * V[next_state])
9         return q_value
10        # END_YOUR_CODE
```

使用computePolicy函数，根据当前的V(s)返回当前的策略

```
1     def computePolicy(V: Dict[StateT, float]) -> Dict[StateT, ActionT]:
2         # Return the policy given V.
3         # Remember the policy for a state is the action that gives the
greatest Q-value.
4         # IMPORTANT: if multiple actions give the same Q-value, choose the
largest action number for the policy.
5         # HINT: We only compute policies for states in stateActions.
6
7         # 1-a
8         # BEGIN_YOUR_CODE
9         # raise Exception("Not implemented yet")
10        policy = {}
11        for state in stateActions:
12            best_action = None
13            best_q = float("-inf")
14            for action in stateActions[state]:
15                q = computeQ(V, state, action)
16                if q > best_q or (q == max_q and action > best_action):
17                    best_q = q
18                    best_action = action
19            policy[state] = best_action
20        return policy
21        # END_YOUR_CODE
```

使用valueIteration函数中的迭代循环，更新V(s)

```

1     print("Running valueIteration...")
2     v = defaultdict(float) # This will return 0 for states not seen
    (handles terminal states)
3     numIters = 0
4     while True:
5         newV = defaultdict(float) # This will return 0 for states not seen
    (handles terminal states)
6         # update V values using the computeQ function above.
7         # repeat until the V values for all states do not change by more
    than epsilon.
8
9         # 1-b
10        # BEGIN_YOUR_CODE
11        # raise Exception("Not implemented yet")
12        delta = 0
13        for state in stateActions:
14            max_q = max(computeQ(V, state, action) for action in
stateActions[state])
15            newV[state] = max_q
16            delta = max(delta, abs(V[state] - newV[state]))
17        # END_YOUR_CODE
18
19        V = newV
20        numIters += 1
21        if delta < epsilon:
22            break
23    V_opt = V
24    print(("valueIteration: %d iterations" % numIters))
25    return computePolicy(V_opt)

```

1.2 Model-based Monte Carlo

借助getAction方法，使用epsilon-greedy策略确定下一步动作

```

1     def getAction(self, state: StateT, explore: bool = True) -> ActionT:
2         """
3         This algorithm will produce an action given a state.
4
5         Here we use the epsilon-greedy algorithm: with probability
|explorationProb|, take a random action.
6         Should return random action if the given state is not in self.pi.
7         The input boolean |explore| indicates whether the RL algorithm is
in train or test time. If it is false (test), we
8         should always follow the policy if available.
9         HINT: Use random.random() (not np.random()) to sample from the
uniform distribution [0, 1]
10        """
11        if explore:
12            self.numIters += 1
13            explorationProb = self.explorationProb
14            if self.numIters < 2e4: # Always explore
15                explorationProb = 1.0
16            elif self.numIters > 1e6: # Lower the exploration probability by a
logarithmic factor.
17                explorationProb = explorationProb / math.log(self.numIters -
100000 + 1)
18

```

```

19         # 2-a
20         # BEGIN_YOUR_CODE
21         # raise Exception("Not implemented yet")
22         if explore and random.random() < explorationProb:
23             return random.choice(self.actions)
24         else:
25             return self.pi.get(state, random.choice(self.actions))
26         # END_YOUR_CODE

```

使用incorporateFeedback方法，根据当前环境的反馈更新我们对环境的建模以及策略

```

1     def incorporateFeedback(self, state: StateT, action: ActionT,
reward: int, nextState: StateT, terminal: bool):
2         """
3         We will call this function with (s, a, r, s'), which is used to
update tCounts and rTotal.
4         For every self.calcValIterEvery steps, runs value iteration after
estimating succAndRewardProb.
5
6         Args:
7             state: StateT, the current state
8             action: ActionT, the action taken in the current state
9             reward: int, the reward received after taking the action
10            nextState: StateT, the next state
11            terminal: bool, whether the episode is terminated
12        """
13
14        self.tCounts[(state, action)][nextState] += 1
15        self.rTotal[(state, action)][nextState] += reward
16
17        if self.numIters % self.calcValIterEvery == 0:
18            # Estimate succAndRewardProb based on self.tCounts and
self.rTotal.
19            # Hint 1: prob(s, a, s') = (counts of transition (s,a) -> s') /
(total transntions from (s,a))
20            # Hint 2: Reward(s, a, s') = (total reward of (s,a) -> s') /
(counts of transition (s,a) -> s')
21            # Then run valueIteration and update self.pi.
22            succAndRewardProb = defaultdict(list) # (state, action) ->
[(nextState, transitionProb, expectedreward)]
23
24            # 2-b
25            # BEGIN_YOUR_CODE
26            # raise Exception("Not implemented yet")
27            for (s, a), transitions in self.tCounts.items():
28                total_transitions = sum(transitions.values())
29                for ns, count in transitions.items():
30                    prob = count / total_transitions
31                    reward = self.rTotal[(s, a)][ns] / count
32                    succAndRewardProb[(s, a)].append((ns, prob, reward))
33
34            self.pi = valueIteration(succAndRewardProb, self.discount)
35            # END_YOUR_CODE

```

1.3 Q-learning

使用getAction方法，用epsilon-greedy策略确定下一步的动作

```
1     def getAction(self, state: StateT, explore: bool = True) -> ActionT:
2         """
3         This algorithm will produce an action given a state.
4         Here we use the epsilon-greedy algorithm: with probability
5         |explorationProb|, take a random action.
6         The input boolean |explore| indicates whether the RL algorithm is
7         in train or test time. If it is false (test), we
8         should always choose the maximum Q-value action.
9         """
10        if explore:
11            self.numIters += 1
12            explorationProb = self.explorationProb
13            if self.numIters < 2e4: # explore
14                explorationProb = 1.0
15            elif self.numIters > 1e5: # Lower the exploration probability by a
16                logarithmic factor.
17                explorationProb = explorationProb / math.log(self.numIters -
18                    100000 + 1)
19            # HINT 1: You can access Q-value with self.Q[state, action]
20            # HINT 2: Use random.random() to sample from the uniform
21            distribution [0, 1]
22
23            # 3-a
24            # BEGIN_YOUR_CODE
25            # raise Exception("Not implemented yet")
26            if explore and random.random() < explorationProb:
27                return random.choice(self.actions)
28            else:
29                q_values = [self.Q[(state, action)] for action in self.actions]
30                max_q = max(q_values)
31                best_actions = [action for action in self.actions if
32                    self.Q[(state, action)] == max_q]
33                return random.choice(best_actions)
34            # END_YOUR_CODE
```

使用incorporateFeedback方法，根据当前环境的反馈更新Q表

```
1     def incorporateFeedback(
2         self, state: StateT, action: ActionT, reward: float, nextState:
3         StateT, terminal: bool
4     ) -> None:
5         """
6         We will call this function with (s, a, r, s'), which you should use
7         to update |Q|.
8         Note that if s' is a terminal state, then terminal will be True.
9         Remember to check for this.
10        You should update the Q values using self.getStepSize()
11        HINT 1: The target V for the current state is a combination of the
12        immediate reward
13        and the discounted future value.
14        HINT 2: V for terminal states is 0
15        """
16
17        # 3-b
18        # BEGIN_YOUR_CODE
```

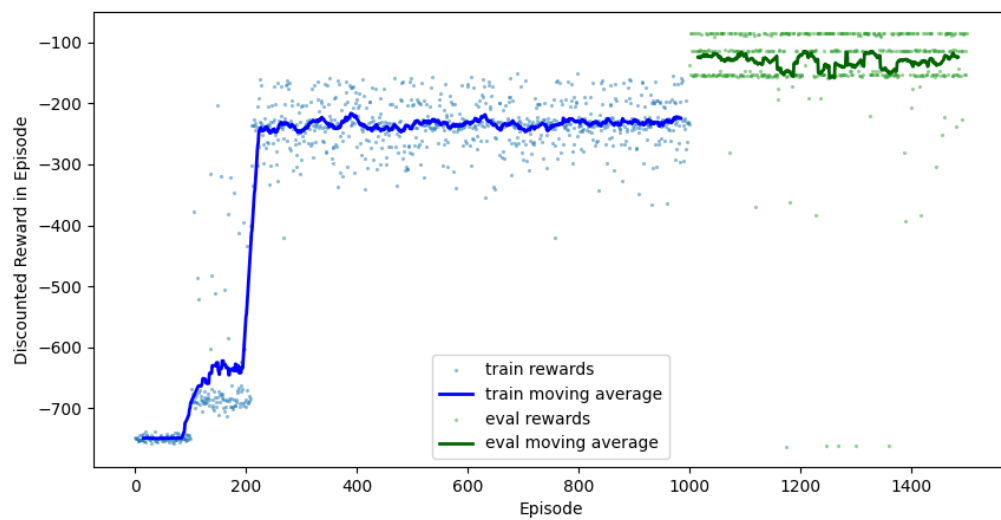
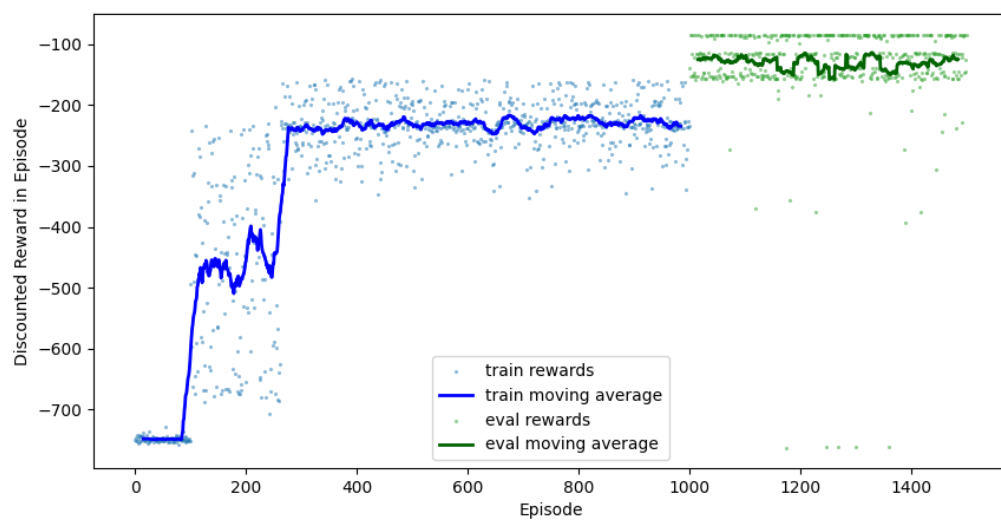
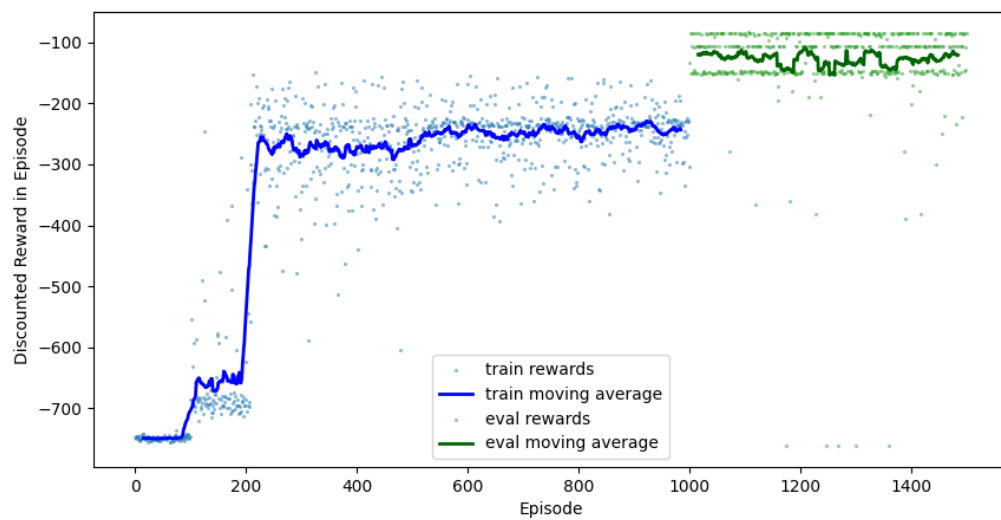
```
15         # raise Exception("Not implemented yet")
16         alpha = self.getStepSize()
17         current_q = self.Q[(state, action)]
18         if terminal:
19             target = reward
20         else:
21             next_q_values = [self.Q[(nextState, a)] for a in self.actions]
22             target = reward + self.discount * max(next_q_values)
23         td_error = target - current_q
24         self.Q[(state, action)] += alpha * td_error
25         # END_YOUR_CODE
```

2 参数回调

本次实验未调试参数就得到了比较好的实验结果

3 最好结果

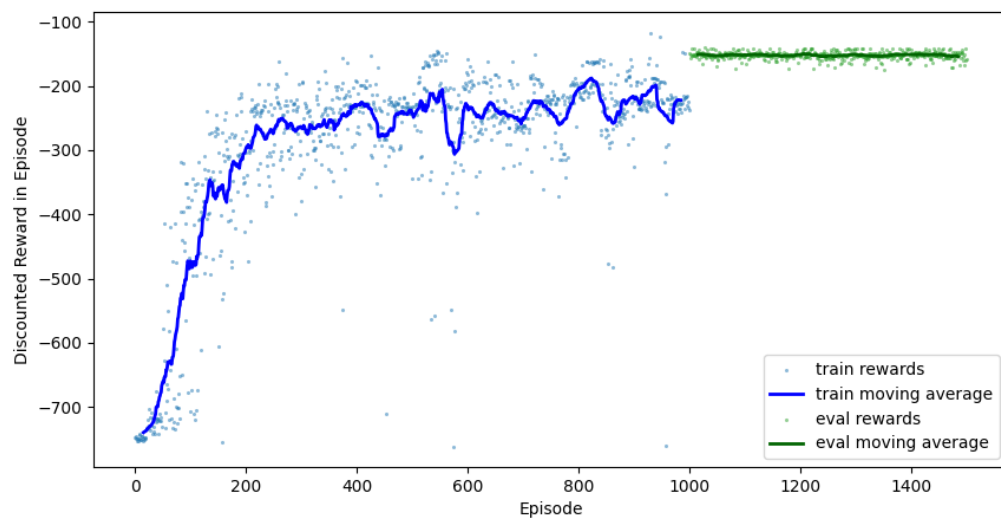
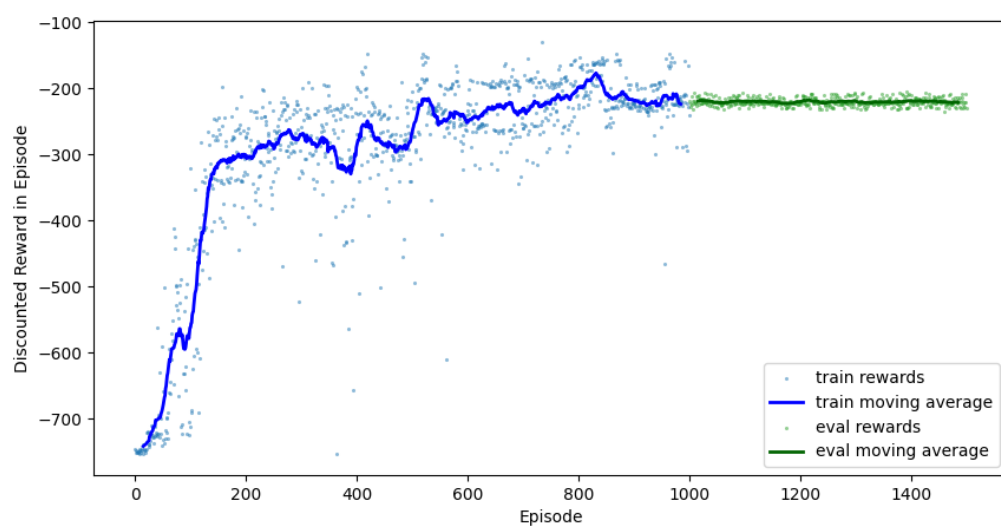
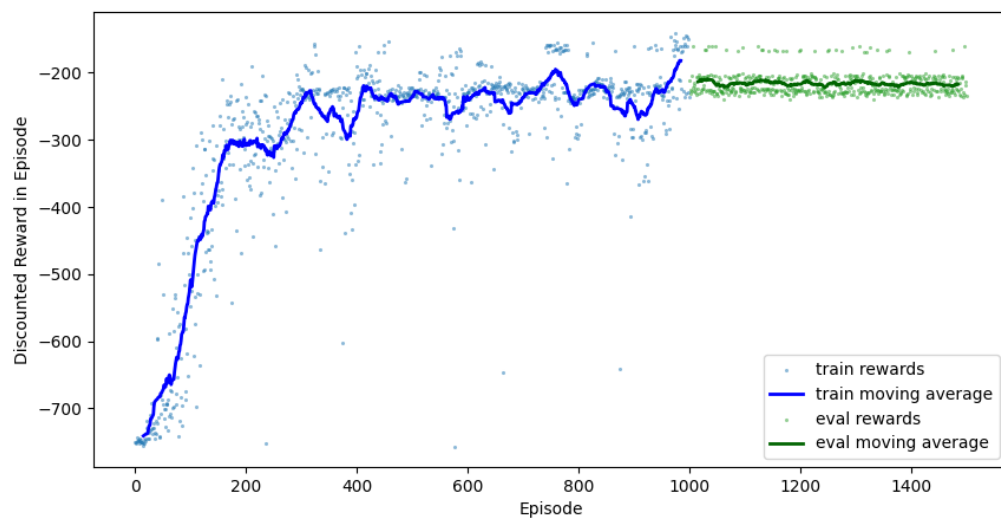
3.1 Model-based Monte Carlo



此次训练得到了较高的一个reward(-227.21501)

```
Trial 3 out of 3
*****
Trial 0 (totalReward = -748.20917, Length = 1000)
Running valueIteration...
valueIteration: 6269 iterations
Trial 100 (totalReward = -690.02386, Length = 1000)
Trial 200 (totalReward = -697.19543, Length = 1000)
Running valueIteration...
valueIteration: 1544 iterations
Trial 300 (totalReward = -205.62802, Length = 200)
Trial 400 (totalReward = -271.25342, Length = 268)
Trial 500 (totalReward = -282.57648, Length = 305)
Trial 600 (totalReward = -171.83987, Length = 160)
Running valueIteration...
valueIteration: 495 iterations
Trial 700 (totalReward = -260.56955, Length = 266)
Trial 800 (totalReward = -223.23991, Length = 223)
Trial 900 (totalReward = -227.21501, Length = 232)
Training complete! Running evaluation, writing weights to
```

3.2 Q-learning



此次训练得到了较高的一个reward(-159.84117)


```
Trial 1 out of 3
```

```
*****
```

```
Trial 0 (totalReward = -751.35657, Length = 1000)
Trial 100 (totalReward = -547.18146, Length = 644)
Trial 200 (totalReward = -276.6558, Length = 278)
Trial 300 (totalReward = -285.66458, Length = 283)
Trial 400 (totalReward = -189.98355, Length = 193)
Trial 500 (totalReward = -232.27441, Length = 236)
Trial 600 (totalReward = -229.84988, Length = 235)
Trial 700 (totalReward = -221.51367, Length = 226)
Trial 800 (totalReward = -226.39867, Length = 229)
Trial 900 (totalReward = -159.84117, Length = 161)
Training complete! Running evaluation, writing weights to
```

3.3 grade

此时使用grade.py评测有较好的分数29.415

```
100%|
100%|
ModelBasedMonteCarlo: -134.88131713867188 +/- 39.574153900146484, Win Rate: 0.961
TabularQLearning: -162.94537353515625 +/- 6.874875068664551, Win Rate: 1.0
You got a score of 29.415 out of 30!
```

4 回答问题

4.1

- 根据输出的图片讨论与分析算法的性能,在更一般的情况下,有哪些因素会导致Model-based Value Iteration算法失效?
- 解答:
 1. 模型自身问题。若环境转移概率或奖励估计偏差过大,会导致迭代结果无法合理收敛;
 2. 环境变化缺乏规律。若环境动态变化或具有非Markov特性,学到的模型可能无法准确预测后续情况;
 3. 折扣因子不合理。过高或过低
- 填空:上面实现的Tabular Q-learning 是一个
Model-free, Temporal Difference, off-Policy, Value-based, online的RL算法。

5 反馈

- 时间:
 - 补全代码: 3h
 - 调试: 1h
 - 写报告: 0.5h